Prompt

1. Brainstorm the spec

- [ ] Prompt: *"Interview me one question at a time to design this feature — cover entities, endpoints, domain rules, auth, and errors — then write the agreed spec to `docs/superpowers/specs/YYYY-MM-DD--design.md` and commit it."*
- [ ] Answer one question at a time: `entities → endpoints → domain rules → auth → errors`
- [ ] Output: `docs/superpowers/specs/YYYY-MM-DD--design.md` (committed)

> ⚠ **Gate You must approve the spec before any code is written.**

2. Write the plan

- [ ] Prompt: *"Turn the approved spec into a bite-sized TDD plan with exact file paths and code for each step, and save it to `docs/superpowers/plans/YYYY-MM-DD-<feature>.md`."*
- [ ] Output: `docs/superpowers/plans/YYYY-MM-DD-<feature>.md` (bite-sized TDD tasks, exact paths + code)
- [ ] Task order:
  1. spec docs
  2. migration
  3. entities
  4. repositories
  5. DTOs
  6. service
  7. controller
  8. error handling
  9. tests
  10. deploy

3. Spec-driven docs (required by backend/CLAUDE.md)

Before any code, update `backend/docs/` **in this order**:

1. [ ] `datamodel.md` (+ Mermaid E-R diagram)
2. [ ] `dataflow.md`
3. [ ] `architecture.md`
4. [ ] `api-contract.md`

> 🖊 **Enforced by the `spring-boot-spec-driven` skill.**

4. Branch

- [ ] Never work on `main`

```
git checkout -b feat/<feature-name>
```

5. Execute the plan (TDD)

- [ ] Prompt: *"Work through the plan task by task: for each one, write a failing test first, run it, implement the minimal code to pass, run it again, then commit before moving to the next task."*

- Per task:
  1. write failing test
  2. run → see it fail
  3. minimal implementation
  4. run → see it pass
  5. commit

6. Verify (green gate)

```
cd backend
./gradlew test            # unit + slice
./gradlew integrationTest # Testcontainers; Docker must be up
```

> 🗲 **Not done until green Fix the root cause — never disable a test.**

7. Finish the branch

- [ ] Prompt: *"All tests are green — review the branch's changes with me, then ask whether I want to merge, open a PR, keep the branch as-is, or discard it."*
- [ ] Choose: `merge` / `PR` / `keep` / `discard`

House rules (always on)

> 🔥 **Conventions to enforce everywhere**

- Thin controllers — no business logic in controllers
- `@Transactional` on services
- `record` DTOs — never expose entities
- Constructor injection
- Central RFC-7807 error handling
- Never edit an applied Flyway migration — add a new `V<n>__…`

Prompt — Generate the Graph Explorer Frontend from the Backend

> **Usage** Copy everything below this line into an agent (e.g. Claude Code) running at the repo root. It builds the `nodeandedges-` React frontend against the existing Spring Boot backend, using the backend's own contract docs as the source of truth.

Task

Build a **catalog-driven graph-exploration frontend** for the read-only graph query API in `backend/`.

The backend is the source of truth: **read these before writing any code and never invent shapes.**

Source	Role
`backend/docs/api_contract.md`	Authoritative REST contract (endpoints, request/response records, status codes, RFC 7807 error bodies, pagination). **Doc wins over code** if they disagree.
`backend/src/main/resources/graph-catalog.yml`	Example data only — UI must be driven by live `GET /api/catalog`, not hardcoded.
`backend/docs/datamodel.md`, `backend/docs/architecture.md`	Background on the model.

> ✕ **Do NOT Hardcode any entity types, attributes, or relationships. Everything the UI renders about the domain comes from `GET /api/catalog` at runtime.**

Stack (match the existing project)

- React 19 + Vite + TypeScript
- `@xyflow/react` (React Flow v12) for the graph canvas
- **Zustand v5** for state
- **Vitest** for unit tests
- Package manager: `npm`
- `tsconfig.app.json`: `erasableSyntaxOnly: true` (no TS enums, no constructor parameter properties) and `noUnusedLocals: true` → plain functions + string-literal union types
- App lives in `nodeandedges-/`; run npm/npx from there

Backend endpoints (verify each against `api_contract.md`)

Base path `/api`, dev port `8080`. Every request except `GET /api/catalog` carries a `backend` field = `"neo4j"` (JSON body field on POSTs, `?backend=neo4j` query param on GETs).

1. `GET /api/catalog`

```
{ nodeTypes:[{id,displayName,attributes:
[{name,displayName,dataType,filterable,displayable,allowedOperators}]}],
  edgeTypes:[{id,displayName,sourceTypeId,targetTypeId,cardinality,direction}]
}
```

Fetch once at startup, cache for the session.

2. `POST /api/nodes/search`

Seed a root.

- Body: `{nodeType, backend, filters?, page:{number,size}, sort:{attribute,direction}}`
- Returns: `{nodes:[{graphNodeId, attributes}], total}`

3. `GET /api/nodes/{id}/edges?nodeType=&backend=`

- Returns: `[{edgeTypeId,targetTypeId,count}]` (per-edge neighbour counts for fan-out warning)

4. `POST /api/nodes/{id}/expand`

Walk one edge from one node.

- Body: `{nodeType(parent), backend, edgeTypeId, direction?, filters?, page, sort}`
- Returns: `{nodes:[{parentId,edgeInstanceId,graphNodeId,attributes}], total}`

5. `POST /api/report`

Flatten a view chain into a table.

- Body: `{backend, view:{steps:[{id,parentId,nodeType,viaEdgeType,direction?,filters?}]}, selectedColumns:[{stepId,attribute}], sort:{stepId,attribute,direction}, page}`
- Returns: `{columns:[{key,nodeType,attribute,displayName}], rows:(string|null)[][], total}`
- A `null` cell = the path never reached that level; **not an error**.

Contract rules to honour exactly

> **Important**
>
> - `page` is `{number,size}`, `number` is **1-based**
> - `sort` is `{attribute,direction}` (`ASC|DESC`), on a `filterable` attribute
> - **Both `page` and `sort` are mandatory** on every paginated call
> - Operators: `EQ NEQ IN CONTAINS GT LT BETWEEN`
>   - Arity: `BETWEEN` exactly 2 values, `IN` ≥ 1, all others exactly 1
>   - An attribute's legal operators come from its `allowedOperators` — build the filter UI from that list, don't hardcode the matrix
> - Errors are RFC 7807 `application/problem+json`. Decode into a typed error carrying `type/title/status/detail` plus any extra fields
> - Surface `409 /problems/fan-out-exceeded` (`{count,limit,edgeTypeId}`) as an actionable **"N results, cap M — add a filter"** message

Architecture to build

API layer (`src/api/`)

The only place that knows the wire format:

- `types.ts` — mirrors `api_contract.md` 1:1 (`CatalogResponse`, `ApiNodeType`, `ApiAttribute`, `Operator`, request/response records, `ProblemDetail`)
- `client.ts` — a `fetch` wrapper
  - Base URL: `import.meta.env.VITE_API_BASE ?? 'http://localhost:8080'`

    - Parses error bodies into a `ProblemError` class (with a `kind` getter that strips the `type` slug)
- `endpoints.ts` — `getCatalog` / `search` / `getEdges` / `expand` / `report`, injecting `backend:"neo4j"` in one place

Catalog-driven domain

A Zustand `catalogStore` loads `GET /api/catalog` at startup and holds it.

A `schema` module exposes accessors reading from the loaded catalog:

- `getEntityType`
- `allEntityTypes`
- `attributesFor`
- `relationshipsFor`
- `defaultAttributes`
- `searchEntityTypes`

> **Catalog shape** The catalog is **flat** (no type inheritance); a type's "relationships" are the `edgeTypes` whose `sourceTypeId` is that type. Data type mapping: `STRING/BOOLEAN → text`, `NUMBER → number`, `DATE → date`.

Adapter (`src/store/adapter.ts`)

The one place the UI model meets the DTOs:

UI operator	API operator
is	`EQ`
is not	`NEQ`
contains	`CONTAINS`
>	`GT`
<	`LT`
between	`BETWEEN`
multi-value / enum selection	`IN`

- Split a `BETWEEN` value into two
- Drop empty filters
- Fold the free-text "search all attributes" box into a `CONTAINS` filter on the label attribute

- Default `sort` to the type's first filterable attribute
- Provide a single `columnFilters(typeId, filters, search)` helper, reused for search **and** expand **and** every report step (so results stay consistent)
- Unit-test the operator/arity mapping with Vitest

Exploration store (`src/store/`)

The client owns the exploration as a **tree of columns**; the server is stateless.

Each column = an entity type + its filters + selected report attributes + the relationship it was reached by.

Actions are async:

- **Root column** → `search()` ; store `instances` (mapped `{graphNodeId, nodeType, label, values, parentId?, edgeInstanceId?}`) and a live count from `total`
- **Child column** → `expand()` from a specific parent instance's `graphNodeId` (per-instance API); a type-level child with no anchored parent stays idle and only contributes to the report
- **Load more** → grows `page.size` and refetches; filter/search changes refetch (debounce keystrokes)
- **Report** → build `view.steps` from the active root→leaf path (root has `parentId` / `viaEdgeType` null, each child names its edge + direction), `selectedColumns` from each column's chosen attributes, call `report()` , render the table (`null` cells → blank)
- **Instance label** = the value of the catalog's first `STRING` attribute for that type — **NOT** the first JSON key (API does not guarantee attribute key order)

Views + on-node controls (`src/components/`)

A React Flow **Graph view** plus other views: Columns / Outline / Radial / Nested / Org-chart / Treemap / Icicle / Matrix.

In the **Graph view**, controls live **on each node** (no left panel):

- Inline **Filter** section (search + per-attribute facets + active-filter chips)
- Inline **Columns** section (report-attribute checkboxes)
- **+ Add** popover (the type's relationships → expand)
- Only one section open at a time

Other requirements:

- Shared `ConfigPanel` (left side) for non-graph views

- **Share the filter/attribute/add components** between the panel and the node so they never drift
- Notes added by double-clicking empty canvas
- Instance detail panel opens on clicking an instance, offers per-instance "reveal related" (anchored expand)

Backend change required

> ⚠ **CORS The SPA runs on a different origin (`http://localhost:5173`) than the API (`:8080`).**

- [ ] Add a single `WebMvcConfigurer` mapping `/api/**`
  - Allowed origins from config property `app.cors.allowed-origins` (dev default `http://localhost:5173`)
  - Methods: `GET,POST,OPTIONS`
  - Header: `Content-Type`
  - Credentials: off
- [ ] Follow `backend/CLAUDE.md` house rules — **update `docs/` (dataflow → architecture → api_contract) before writing the config**
- [ ] Add a slice test asserting the preflight `Access-Control-Allow-Origin` header

Config & verification

- [ ] `.env` / `.env.example` with `VITE_API_BASE=http://localhost:8080` (git-ignore `.env`)

Green gates

```
npx tsc -b --noEmit
npm run build
npm run lint
npm test   # Vitest
```

Live smoke test

With the stack up:

```
docker compose up -d
./gradlew bootRun   # in backend/
npm run dev
```

Checklist:

- [ ] Catalog type-ahead offers the seeded types
- [ ] Searching a root returns real rows
- [ ] A full expand chain flattens through `/api/report` (with `null` cells for short paths)
- [ ] Zero browser console errors (also proves CORS)

> **Prefer a Playwright `smoke.mjs` script to drive this end-to-end.**

Guardrails

> **Non-negotiables**
>
> - Do not invent request/response shapes — read `api_contract.md` and match it exactly
> - Do not hardcode the domain — drive it from `GET /api/catalog`
> - Keep the wire format isolated in `src/api/`; keep the model↔DTO mapping isolated in the adapter
> - Small, focused files; DRY the filter/attribute/add UI across the node and the panel

---

Related Notes

- Spring Boot House Rules
- Graph Explorer Frontend Architecture
- RFC 7807 Error Handling
- Superpowers Workflow Commands